Contents

1 PHP Functions, more advanced topics

Let's delve into the more advanced aspects of PHP functions, including anonymous functions, variable scope, and recursive functions. These concepts allow for more dynamic and flexible programming in PHP.

1.1 Anonymous Functions

Anonymous functions, are functions without a specified name. They are most useful as the value of callback parameters, but they have many other uses.

- **Syntax and Usage**

Anonymous functions are declared with the `function` keyword, followed by a set of parentheses to accept parameters and curly braces to define the function body.

```php
<?=
$greet = function($name) {
    echo "Hello, $name!";
};

$greet("World"); // Outputs: Hello, World!
```

- **Features of Anonymous Functions**
- **Variable Inheritance**: Anonymous functions have the ability to inherit variables from the parent scope. This is achieved using the `use` keyword.

```php
<?=
$message = 'Hello';
$example = function() use ($message) {
    echo $message;
};
$example(); // Outputs: Hello
```

- **Callbacks**: They are often used as callback functions in array operations, such as `array_map` or `array_filter`.

```php
<?=
$numbers = [1, 2, 3, 4, 5];
$squared = array_map(function($n) { return $n * $n; }, $numbers);
print_r($squared); // Outputs: Array ( [0] => 1 [1] => 4 [2] => 9
                   // [3] => 16 [4] => 25 )
```

1.1.1 Closures

A closure is an anonymous function that can capture variables local to the scope it was created in. In PHP, all anonymous functions are closures and they are implemented using the Closure class. They can specify variables to be captured with a use clause in the function header.

```php
<?=
 $x = 1, $y = 2;
 $myClosure = function($z) use ($x, $y)
 {
   return $x + $y + $z;
 };
 echo $myClosure(3); // "6"
```

1.2 Variable Scope

1.2.1 Definition

Variable scope determines the visibility and accessibility of a variable within a PHP program.

1.2.2 *Basic* Scopes

PHP has three basic scopes:

- **Local scope:** Variables declared within a function are local to that function and not accessible outside of the function. This means that the `sum` variable defined in the `calculateSum()` function cannot be accessed outside of the function.

```php
<?=
function calculateSum($num1, $num2) {
```

```
    $sum = $num1 + $num2;
    echo "The sum is: $sum";
}

$result = calculateSum(10, 20);
echo $result;// This will output an error because 'sum'
             // is not defined outside the function
```

- **Global scope:** Variables declared outside of any function are global and can be accessed from anywhere in the program. This means that the `age` variable declared outside of any functions can be accessed from within the `printAge()` function.

```
<?php
$age = 30;

function printAge() {
    echo "Your age is: $age";
}

printAge(); // This will print "Your age is: 30"
```

- **Superglobal variables:** Superglobal variables are a special type of variable that is available globally, even within functions. They are used to retrieve data from the request sent by the client (browser) to the server. Some of the most common superglobal variables include:
  - `$_GET`: Contains data from the `$_GET` superglobal variable, which is used to pass data to the script through the URL query string.
  - `$_POST`: Contains data from the `$_POST` superglobal variable, which is used to pass data to the script through the form POST method.
  - `$_REQUEST`: Contains data from both the `$_GET` and `$_POST` superglobal variables, as well as additional data from other methods such as PUT, DELETE, and PATCH.
  - `$_SERVER`: Contains information about the server environment, such as the client's IP address, the request method, and the server's protocol.
  - `$_FILES`: Contains information about uploaded files, such as the file name, file type, and file size.
  - `$_COOKIE`: Contains data from cookies that are sent by the script to the client's browser and stored on the client's computer.

1.2.3 Advanced Scope Concepts

In addition to the basic scopes, there are a few more advanced scope concepts in PHP:

- **Static scope:** Static variables maintain their value between function calls. They are declared within a function using the `static` keyword.

```
<?=
function testStatic() {
```

```
    static $z = 0;
    $z++;
    echo $z;
}
testStatic(); // Outputs: 1
testStatic(); // Outputs: 2
```

- **Closure scope:** Closures are functions that can capture variables local to the scope they were created in. In PHP, all anonymous functions are closures and they are implemented using the Closure class. They can specify variables to be captured with a `use` clause in the function header.

```
<?=
function testClosure() {
    $x = 1, $y = 2;
    $myClosure = function($z) use ($x, $y) {
        return $x + $y + $z;
    };
    echo $myClosure(3); // "6"
}
testClosure();
```

- **Dynamic scope:** Dynamic scope is a feature of PHP that allows variables to be accessed from scopes other than their own. This is typically done with the `$GLOBALS` array, but it can also be done with other techniques such as `eval()` and `extract()`. Dynamic scope can be dangerous because it can lead to unexpected behavior, so it is generally not recommended.

1.2.4 Conclusion

Understanding variable scope is essential for writing good, maintainable PHP code. It is important to be aware of the different scopes and how they interact with each other. By using the right scope for your variables, you can make your code more readable, reusable, and less prone to errors.

1.3 Recursive Functions

A recursive function is a function that calls itself during its execution. This technique is best used for solving problems that can be broken down into simpler, repetitive tasks.

1.3.1 Example of a Recursive Function

A classic example is the calculation of the factorial of a number.

```
<?=
  function factorial($n) {
      if ($n == 0) {
          return 1;
      } else {
          return $n * factorial($n - 1);
      }
```

```
}

echo factorial(5); // Outputs: 120
```

In this example, `factorial()` calls itself with a decremented value until it reaches the base case (`$n` = 0=).

1.3.2 Conclusion

Understanding these advanced aspects of PHP functions opens up a myriad of possibilities for writing more efficient and effective code. Anonymous functions provide flexibility, especially in functional programming and event-driven programming. Understanding variable scope is crucial for managing data within and outside functions, while recursive functions are essential for solving certain types of iterative problems efficiently.

With these tools in your PHP arsenal, you are well-equipped to tackle more complex programming challenges and to write code that is both powerful and elegant.

2 Functions other advanced topics

In recent versions of PHP, two significant additions to the language are arrow functions and generators. These features enhance the functionality and readability of PHP code, particularly in scenarios involving callbacks and iterative operations.

2.1 Arrow Functions

Introduced in PHP 7.4, arrow functions provide a more concise syntax for writing anonymous functions. They are similar to anonymous functions (closures) but with a simpler syntax that's useful for defining simple callback functions.

2.1.1 Syntax

Arrow functions use the `fn` keyword. The basic syntax is:

```
<?=
$arrowFunction = fn($arguments) => $expression;
```

2.1.2 Characteristics

- **Concise Syntax**: Arrow functions allow for a more compact syntax compared to traditional anonymous functions.
- **Automatic Variable Inheritance**: Variables used inside the arrow function are automatically captured by value from the parent scope.

2.1.3 Example

Using an arrow function to square numbers in an array:

```php
<?=
$numbers = [1, 2, 3, 4, 5];
$squared = array_map(fn($n) => $n * $n, $numbers);
// $squared = [1, 4, 9, 16, 25]
```

2.2 Generators

Generators, introduced in PHP 5.5, are a special type of function that allow you to iterate through data without needing to create an array in memory. They are particularly useful for handling large datasets or streams of data.

2.2.1 Syntax

A generator is defined like a regular function, but instead of returning a value, it yields values using the `yield` keyword.

```php
<?=
function myGenerator() {
    yield 'value1';
    yield 'value2';
    // ...
}
```

2.2.2 Characteristics

- **Memory Efficiency**: Instead of loading an entire dataset into memory, a generator yields one value at a time, making it more memory-efficient.
- **State Preservation**: Generators remember their state between successive calls.

2.2.3 Example

Using a generator to iterate over a range of numbers:

```php
<?=
function generateNumbers($start, $end) {
    for ($i = $start; $i <= $end; $i++) {
        yield $i;
    }
}

foreach (generateNumbers(1, 5) as $number) {
    echo $number . ' ';
}
// Output: 1 2 3 4 5
```

2.3 Conclusion

Arrow functions and generators are powerful additions to PHP, enhancing the language's capabilities for functional programming and data iteration. Arrow functions provide a more concise and readable syntax for small functions or callbacks, while generators offer an efficient way to handle large datasets or streams, conserving memory and processing power.